LAPORAN PRAKTIKUM EKSPLORASI HDFS
BLOCK, REPLICATION, DAN DATA LOCALITY



Dosen Pengampu:

Ersha Aisyah Elfaiz, S.Si., M.T.I.

Disusun oleh:

Muhammad Anwar Ismail (24051204161)

Program Studi Teknik Informatika

Fakultas Teknik

Universitas Negeri Surabaya

2026

DAFTAR ISI

DAFTAR ISI ..... **i**
DAFTAR GAMBAR **1**
BAB I PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Tujuan Praktikum 3
BAB II DASAR TEORI **4**
2.1 Hadoop Distributed File System (HDFS) 4
2.2 Block HDFS 4
2.3 Replication Factor dan Replica 4
2.4 HDFS fsck 4
2.5 YARN 4
2.6 MapReduce 5
2.7 Data Locality 5
BAB III PEMBAHASAN **6**
3.1 Hasil dan Tangkapan Layar 6
3.2 Rekapitulasi Block bigfile.txt 18
3.3 Catatan Ukuran bigfile.txt 19
3.4 Hubungan DataNode, NodeManager, Mapper, Reducer, dan ResourceManager 19
3.5 Alasan Replica Tidak Berarti Data Diproses Dua Kali 19
BAB IV KESIMPULAN **21**
4.1 Kesimpulan 21
LAMPIRAN A COMMAND UTAMA **22**
LAMPIRAN B PORT YANG DIGUNAKAN **27**

DAFTAR GAMBAR

Gambar 3.1.1 - Output perintah docker ps untuk memverifikasi status container Hadoop yang sedang aktif dan berjalan.png 6
Gambar 3.1.2 - Output hdfs dfsadmin -report yang menunjukkan terdapat 2 DataNode yang berstatus aktif_1.png 7
Gambar 3.1.3 - Output hdfs dfsadmin -report yang menunjukkan terdapat 2 DataNode yang berstatus aktif_2.png 7
Gambar 3.1.4 - Pengecekan nilai default Replication Factor pada konfigurasi cluster Hadoop.png 8
Gambar 3.1.5 - Output perintah dir untuk melihat ukuran asli file input.txt di penyimpanan lokal Windows sebelum diunggah ke HDFS.png 8
Gambar 3.1.6 - Output hdfs dfs -ls yang membuktikan file input.txt telah tersimpan di HDFS dengan 2 replica.png 9
Gambar 3.1.7 - Laporan HDFS fsck untuk input.txt yang memperlihatkan ID block dan letak DataNode yang menyimpan setiap salinannya_1.png 10
Gambar 3.1.8 - Laporan HDFS fsck untuk input.txt yang memperlihatkan ID block dan letak DataNode yang menyimpan setiap salinannya_2.png 10
Gambar 3.1.9 - Output perintah dir yang menunjukkan ukuran asli bigfile.txt (mendekati 5 MB) pada penyimpanan lokal Windows.png 11
Gambar 3.1.10 - Pengecekan ukuran default block HDFS (128 MB) menggunakan hdfs getconf.png 11
Gambar 3.1.11 - Laporan HDFS fsck untuk bigfile.txt yang memperlihatkan file terpecah menjadi 5 block secara terdistribusi pada 2 DataNode (akibat limitasi block size 1 MB)_1.png 12
Gambar 3.1.12 - Laporan HDFS fsck untuk bigfile.txt yang memperlihatkan file terpecah menjadi 5 block secara terdistribusi pada 2 DataNode (akibat limitasi block size 1 MB_2.png..... 13
Gambar 3.1.13 - Laporan HDFS fsck untuk bigfile.txt yang memperlihatkan file terpecah menjadi 5 block secara terdistribusi pada 2 DataNode (akibat limitasi block size 1 MB_3.png..... 13

Gambar 3.1.14 - mendokumentasikan Application ID dan status penyelesaian (SUCCEEDED) dari job MapReduce WordCount_1.png 15
Gambar 3.1.15 - mendokumentasikan Application ID dan status penyelesaian (SUCCEEDED) dari job MapReduce WordCount_2.png 15
Gambar 3.1.16 - mendokumentasikan Application ID dan status penyelesaian (SUCCEEDED) dari job MapReduce WordCount_3.png 16
Gambar 3.1.17 - Antarmuka YARN UI.png 17
Gambar 3.1.18 - Output akhir dari job MapReduce WordCount di HDFS menggunakan perintah hdfs dfs -cat.png 18

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pertumbuhan data yang sangat cepat menuntut sistem penyimpanan yang mampu menampung berkas berukuran besar sekaligus tetap aman ketika salah satu server mengalami gangguan. Hadoop Distributed File System (HDFS) menjawab kebutuhan tersebut dengan memecah setiap berkas menjadi block, menyalin block ke beberapa DataNode, dan mencatat lokasinya pada NameNode. Mekanisme ini menjadi fondasi bagi pemrosesan paralel oleh MapReduce di atas YARN.

Praktikum 1 dan 2 telah membahas dasar HDFS serta MapReduce WordCount. Praktikum 3 melanjutkan kedua materi itu dengan mengamati langsung bagaimana berkas dipecah menjadi block, berapa replica yang terbentuk, dan DataNode mana yang menyimpannya. Pengamatan tersebut kemudian dihubungkan dengan konsep data locality saat job WordCount dijalankan pada klaster Hadoop 3.4.3 berbasis Docker yang terdiri atas satu NameNode, dua DataNode, satu ResourceManager, dan dua NodeManager.

1.2 Tujuan Praktikum

1. Memahami hubungan antara file, block, replica, dan DataNode pada HDFS.
2. Mengamati lokasi block dan replica menggunakan perintah `hdfs fsck`.
3. Memahami fungsi replication factor sebagai penunjang fault tolerance.
4. Membandingkan jumlah block pada file kecil (input.txt) dan file yang lebih besar (bigfile.txt).
5. Memahami konsep data locality pada pemrosesan MapReduce.
6. Menghubungkan penyimpanan HDFS dengan eksekusi MapReduce melalui YARN.

BAB II
DASAR TEORI

2.1 Hadoop Distributed File System (HDFS)

HDFS adalah sistem berkas terdistribusi yang dirancang untuk menyimpan data berukuran besar pada sekumpulan mesin komoditas. Arsitekturnya menggunakan pola master dan worker. NameNode bertugas sebagai master yang menyimpan metadata berupa struktur direktori, daftar block, dan lokasi block, sedangkan DataNode bertugas sebagai worker yang menyimpan isi block secara fisik.

2.2 Block HDFS

Block adalah satuan penyimpanan terkecil pada HDFS. Ukuran default block pada Hadoop 3.x adalah 128 MB, sebagaimana terbaca pada nilai dfs.blocksize sebesar 134217728 byte di klaster praktikum ini. Berkas yang lebih kecil dari ukuran block tetap menjadi satu block dan hanya memakai ruang sebesar isi aslinya, sedangkan berkas yang lebih besar dipecah menjadi beberapa block.

2.3 Replication Factor dan Replica

Replication factor menentukan jumlah salinan setiap block yang dipertahankan HDFS. Salinan tersebut disebut replica dan disebar ke DataNode yang berbeda. Jika satu DataNode gagal, HDFS masih dapat membaca data dari replica pada DataNode lain, sehingga replica berperan sebagai jaminan ketahanan terhadap kegagalan node.

2.4 HDFS fsck

hdfs fsck adalah perintah untuk memeriksa kesehatan berkas pada HDFS. Dengan opsi -files -blocks -locations, perintah ini menampilkan ukuran berkas, jumlah block, ID setiap block, jumlah replica aktif, dan alamat DataNode yang menyimpan masing-masing replica.

2.5 YARN

Yet Another Resource Negotiator (YARN) mengelola sumber daya dan penjadwalan aplikasi pada klaster. ResourceManager menerima job dan membagi

sumber daya, sedangkan NodeManager pada setiap worker menjalankan dan memantau container tempat task berjalan.

2.6 MapReduce

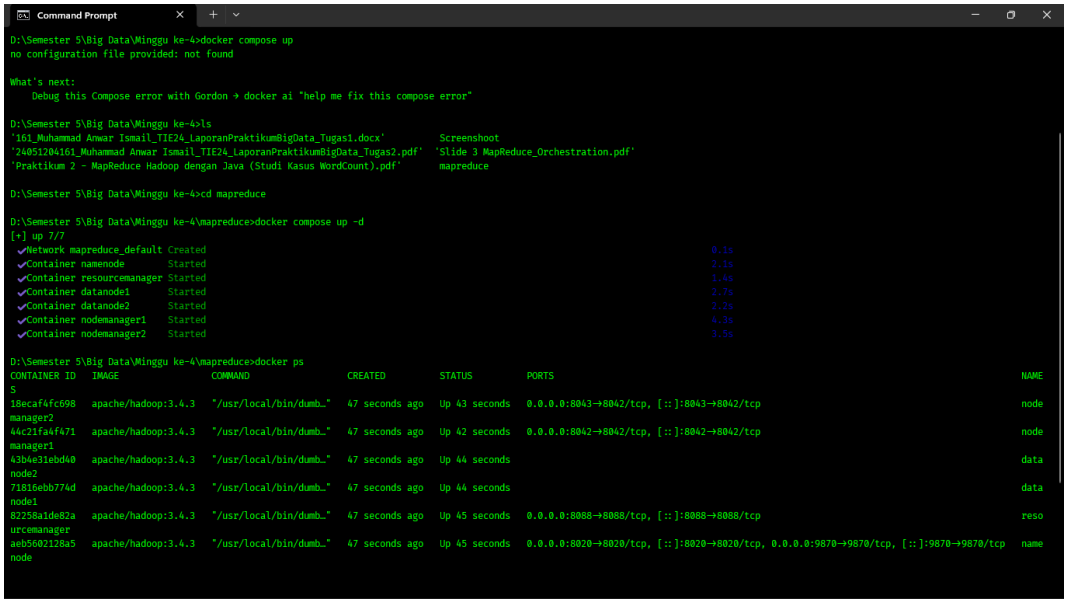
MapReduce adalah model pemrosesan data terdistribusi yang terdiri atas tahap Map, Shuffle and Sort, dan Reduce. Setiap Mapper memproses satu input split, biasanya sebesar satu block HDFS, lalu Reducer menggabungkan hasil dari seluruh Mapper. Pada WordCount, Mapper memecah teks menjadi kata dan Reducer menjumlahkan kemunculan setiap kata.

2.7 Data Locality

Data locality adalah prinsip Hadoop untuk menjalankan task sedekat mungkin dengan lokasi data inputnya. Scheduler lebih memilih node yang menyimpan block, kemudian node lain dalam rak yang sama, sebelum memilih node di rak berbeda. Pendekatan ini menekan perpindahan data melalui jaringan dan mempercepat pemrosesan.

BAB III
PEMBAHASAN

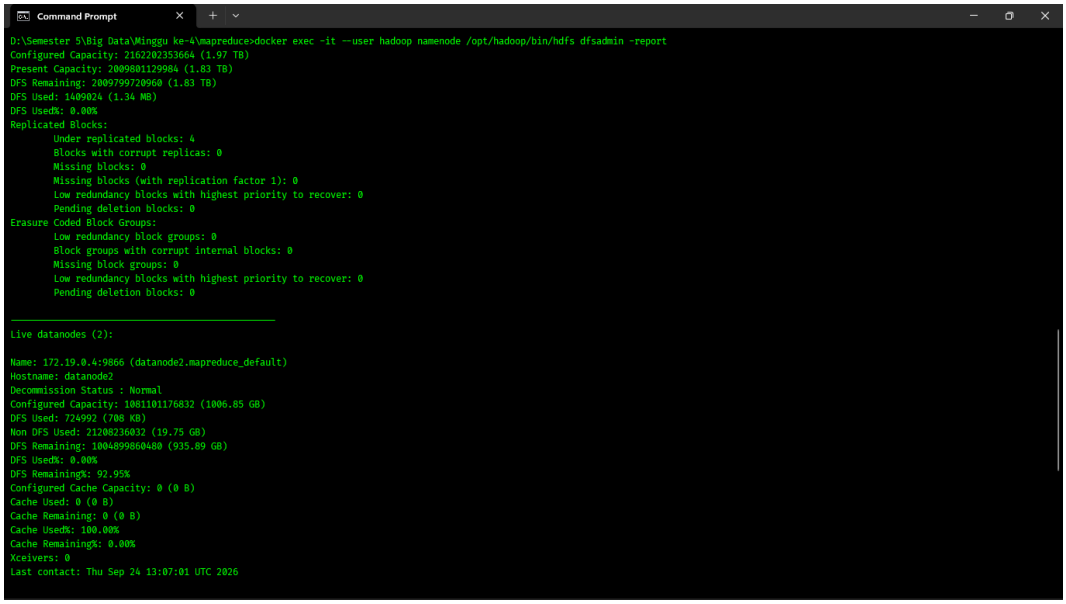
3.1 Hasil dan Tangkapan Layar



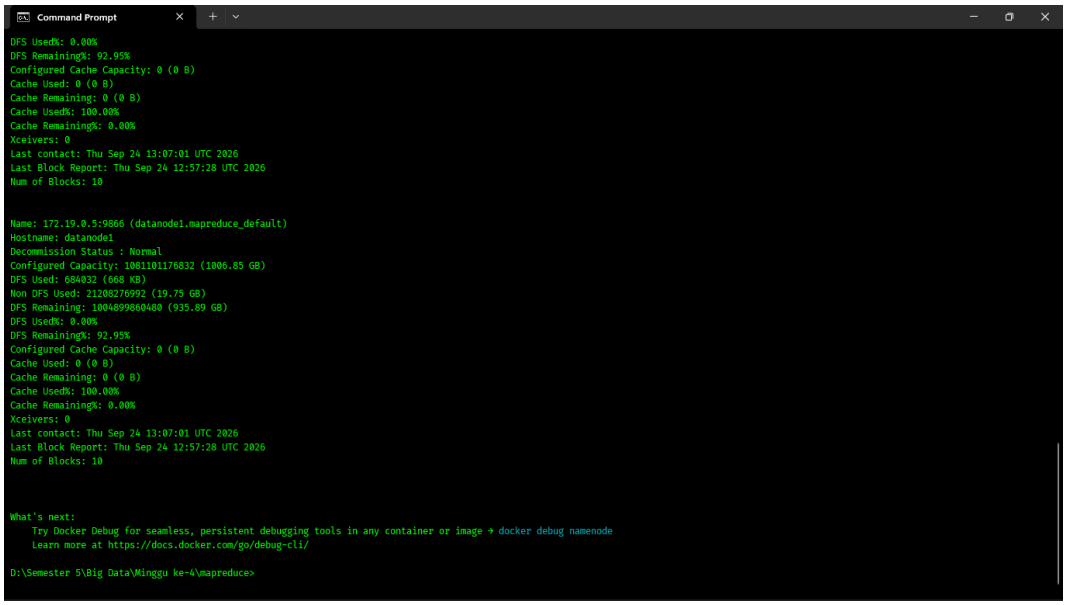
Gambar 3.1.1 - Output perintah docker ps untuk memverifikasi status container Hadoop yang sedang aktif dan berjalan.png

```
docker ps
```

Perintah `docker compose up -d` berhasil menjalankan jaringan dan enam container, yaitu NameNode, ResourceManager, dua DataNode, dan dua NodeManager. Hasil `docker ps` memperlihatkan seluruh container berstatus Up dengan port 8042, 8043, 8088, 8020, dan 9870 terbuka ke host.



Gambar 3.1.2 - Output hdfs dfsadmin -report yang menunjukkan terdapat 2 DataNode yang berstatus aktif_1.png

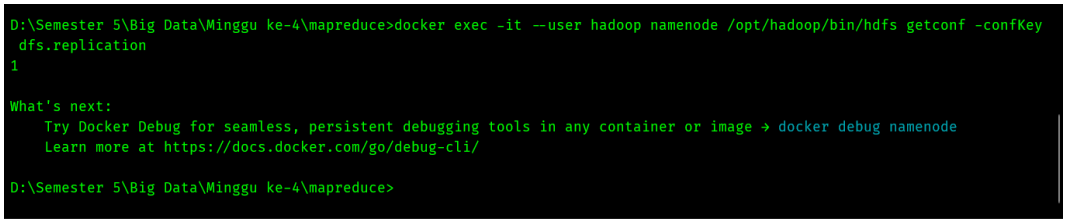


Gambar 3.1.3 - Output hdfs dfsadmin -report yang menunjukkan terdapat 2 DataNode yang berstatus aktif_2.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report
```

Laporan dfsadmin -report menunjukkan Live datanodes berjumlah 2, yaitu datanode2 (172.19.0.4) dan datanode1 (172.19.0.5), keduanya berstatus Normal.

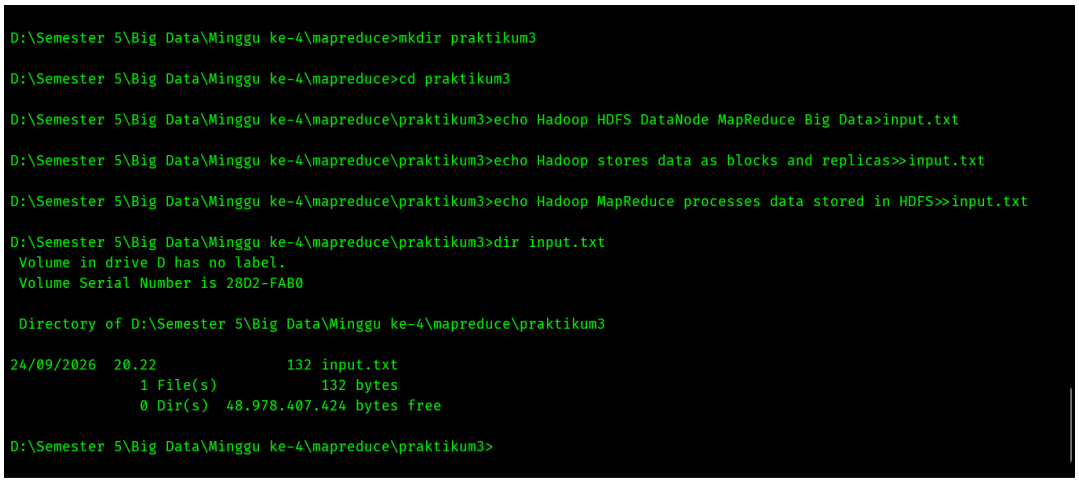
Kapasitas terkonfigurasi cluster mencapai 1,97 TB dengan jumlah missing blocks dan corrupt replicas sebesar 0. Bagian lanjutan laporan memperlihatkan kondisi datanode1 dengan kapasitas 1006,85 GB dan status Normal. Masing-masing DataNode mencatat 10 block, sehingga kedua node terbukti aktif dan sama-sama menampung data.



Gambar 3.1.4 - Pengecekan nilai default Replication Factor pada konfigurasi cluster Hadoop.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs getconf -confKey dfs.replication
```

Nilai dfs.replication pada cluster adalah 1, sehingga file yang diunggah tanpa opsi tambahan hanya memiliki satu replica. Oleh sebab itu, pengunggahan berikutnya menggunakan opsi -Ddfs.replication=2 agar setiap block tersimpan pada dua DataNode.

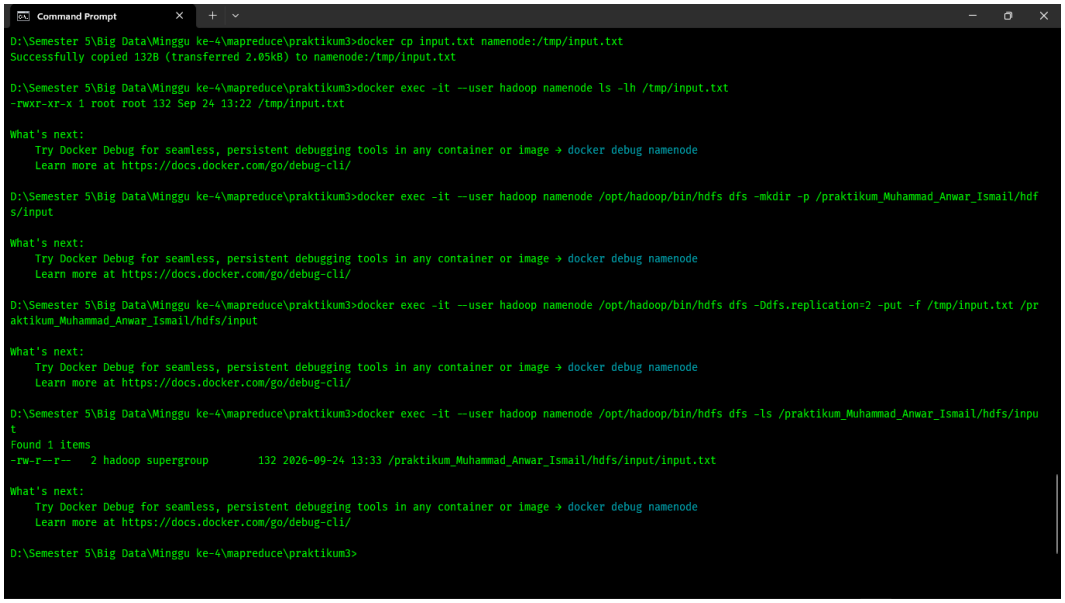


Gambar 3.1.5 - Output perintah dir untuk melihat ukuran asli file input.txt di penyimpanan lokal Windows sebelum diunggah ke HDFS.png

```
mkdir praktikum3
cd praktikum3
```

```
echo Hadoop HDFS DataNode MapReduce Big Data>input.txt
echo Hadoop stores data as blocks and replicas>>input.txt
echo Hadoop MapReduce processes data stored in HDFS>>input.txt
```

Tiga perintah echo membuat file input.txt berisi tiga baris teks, dan perintah dir menampilkan ukurannya sebesar 132 bytes. Ukuran tersebut jauh di bawah block size HDFS, sehingga file ini hanya akan menempati satu block.

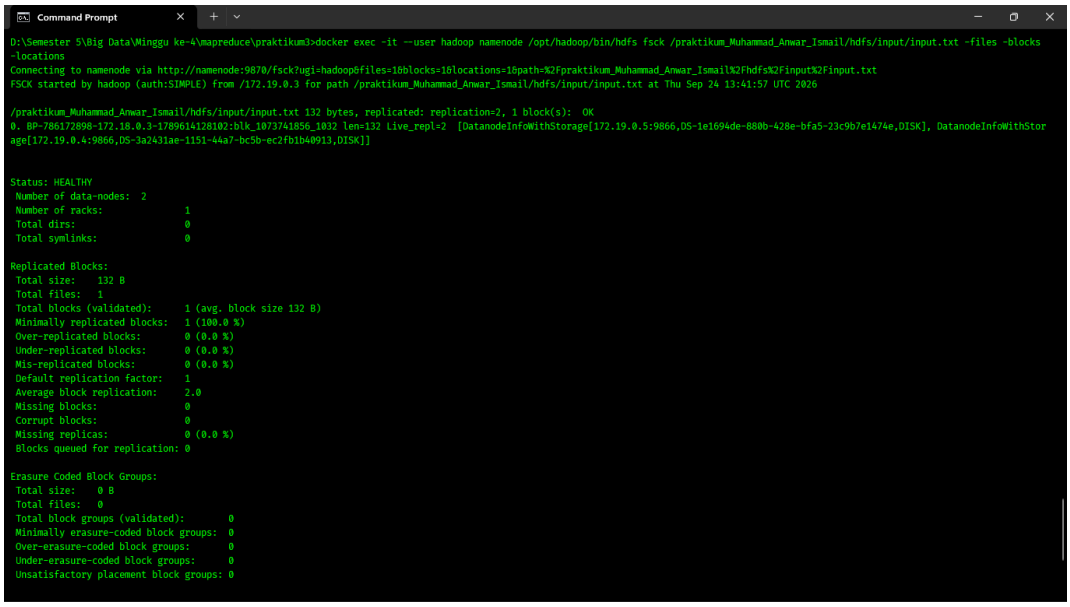


Gambar 3.1.6 - Output hdfs dfs -ls yang membuktikan file input.txt telah tersimpan di HDFS dengan 2 replica.png

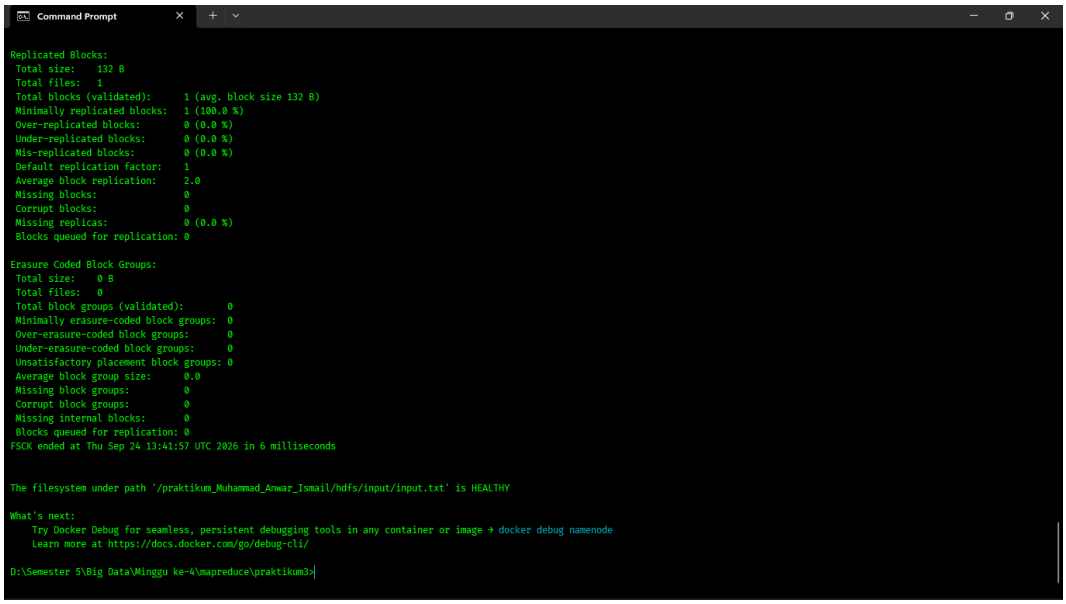
```
docker cp input.txt namenode:/tmp/input.txt
docker exec -it --user hadoop namenode ls -lh /tmp/input.txt
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum_Muhammad_Anwar_Ismail/hdfs/input
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -Ddfs.replication=2 -put -f /tmp/input.txt /praktikum_Muhammad_Anwar_Ismail/hdfs/input
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum_Muhammad_Anwar_Ismail/hdfs/input
```

File input.txt disalin ke container NameNode sebesar 132 B, kemudian diunggah ke direktori HDFS dengan replication factor 2. Hasil hdfs dfs -ls

menampilkan angka 2 pada kolom replication, yang membuktikan file tersebut memiliki dua replica.



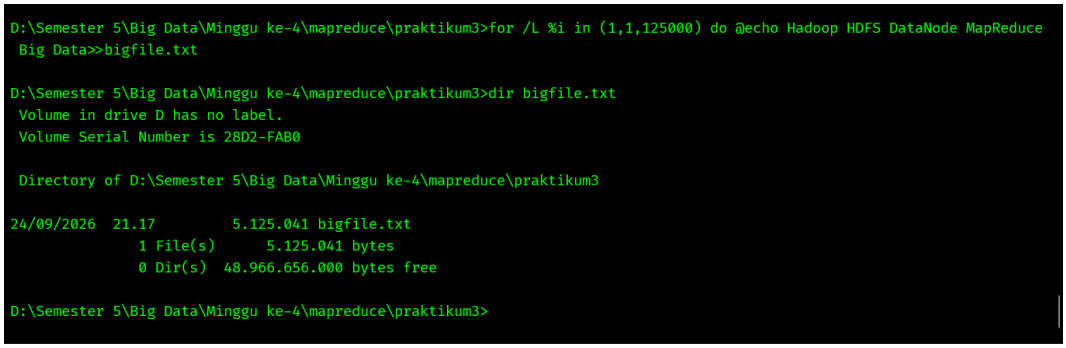
Gambar 3.1.7 - Laporan HDFS fsck untuk input.txt yang memperlihatkan ID block dan letak DataNode yang menyimpan setiap salinannya_1.png



Gambar 3.1.8 - Laporan HDFS fsck untuk input.txt yang memperlihatkan ID block dan letak DataNode yang menyimpan setiap salinannya_2.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs fsck /praktikum_Muhammad_Anwar_Ismail/hdfs/input/input.txt -files -blocks -locations
```

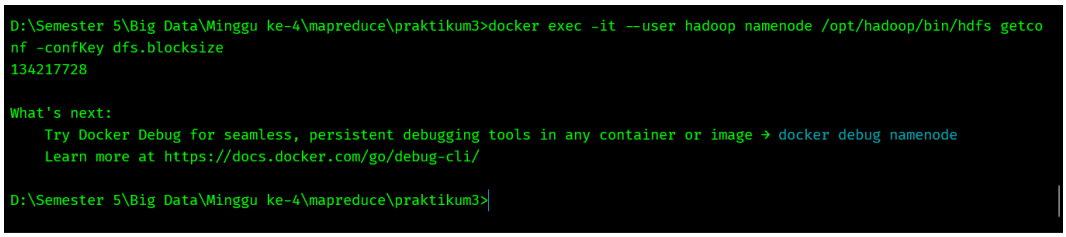
Laporan fsck menyatakan input.txt berukuran 132 bytes dan terdiri atas 1 block dengan ID blk_1073741856_1032 serta Live_repl=2. Kedua replica berada pada DataNode 172.19.0.5 (datanode1) dan 172.19.0.4 (datanode2) dengan status HEALTHY. Ringkasan fsck mencatat Total blocks sebanyak 1 dengan Average block replication bernilai 2.0, walaupun default replication factor cluster tetap 1. Tidak ada missing, corrupt, maupun under-replicated blocks, dan sistem berkas pada path input.txt dinyatakan HEALTHY.



Gambar 3.1.9 - Output perintah dir yang menunjukkan ukuran asli bigfile.txt (mendekati 5 MB) pada penyimpanan lokal Windows.png

```
for /L %i in (1,1,125000) do @echo Hadoop HDFS DataNode MapReduce Big Data>>bigfile.txt
dir bigfile.txt
```

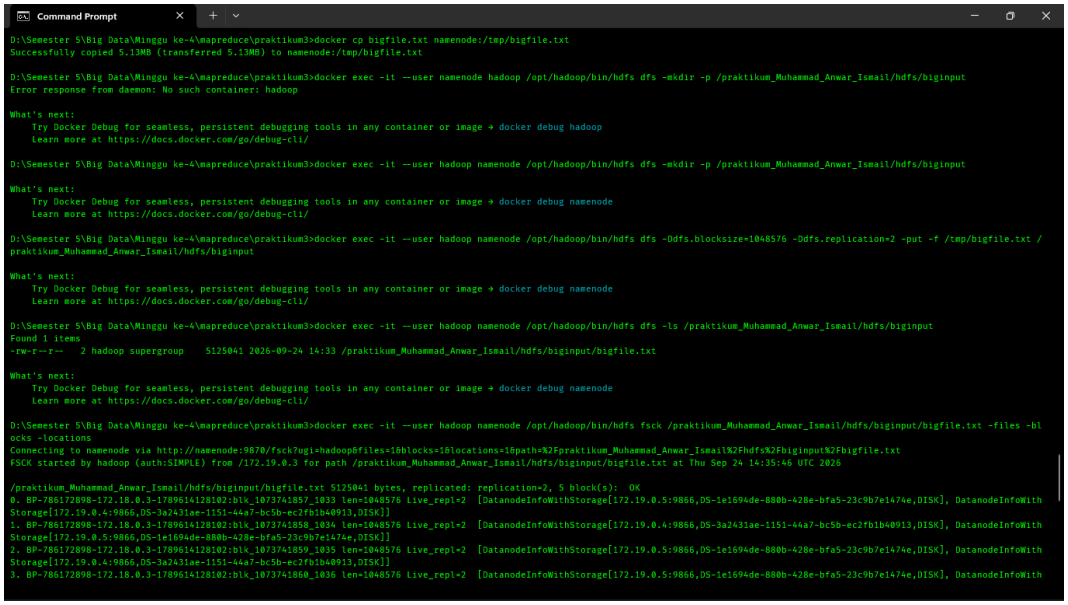
Perulangan for /L menghasilkan file bigfile.txt dengan 125.000 baris teks. Perintah dir mencatat ukuran aktualnya sebesar 5.125.041 bytes, yang mendekati target 5 MB.



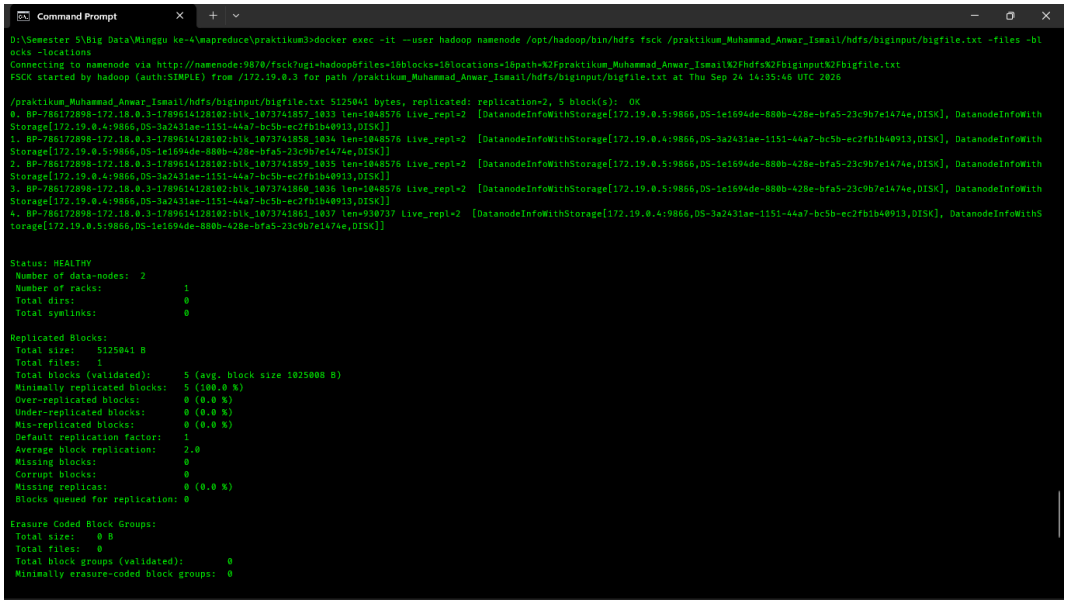
Gambar 3.1.10 - Pengecekan ukuran default block HDFS (128 MB) menggunakan hdfs getconf.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs getconf -confKey dfs.blocksize
```

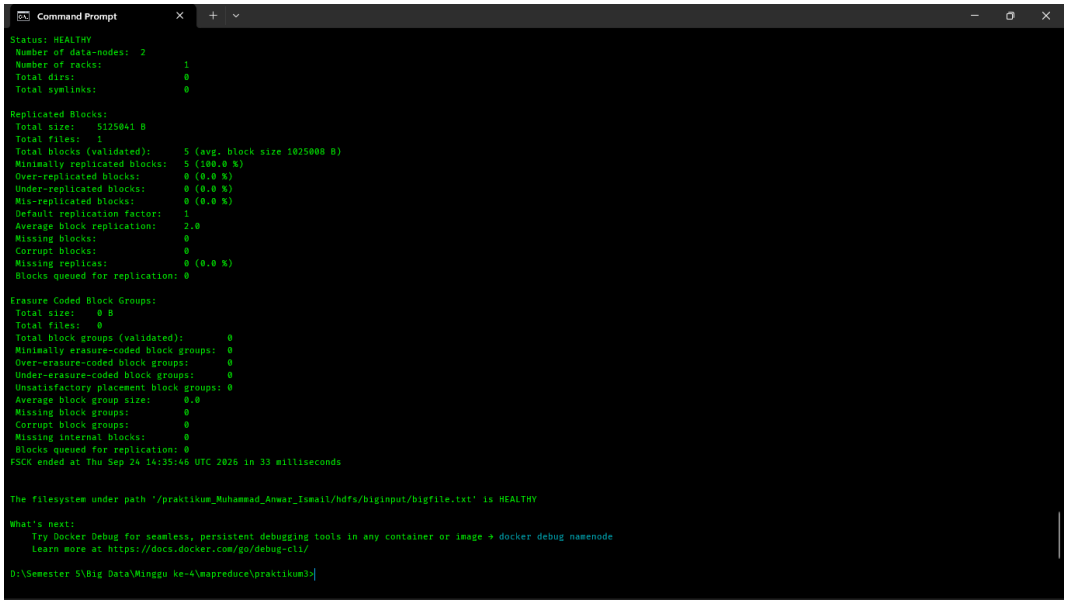
Perintah hdfs getconf menampilkan nilai dfs.blocksize sebesar 134217728 byte, atau 128 MB. Karena bigfile.txt hanya sekitar 5 MB, file tersebut akan menjadi satu block saja jika block size tidak diubah.



Gambar 3.1.11 - Laporan HDFS fsck untuk bigfile.txt yang memperlihatkan file terpecah menjadi 5 block secara terdistribusi pada 2 DataNode (akibat limitasi block size 1 MB)_1.png



Gambar 3.1.12 - Laporan HDFS fsck untuk bigfile.txt yang memperlihatkan file terpecah menjadi 5 block secara terdistribusi pada 2 DataNode (akibat limitasi block size 1 MB)_2.png

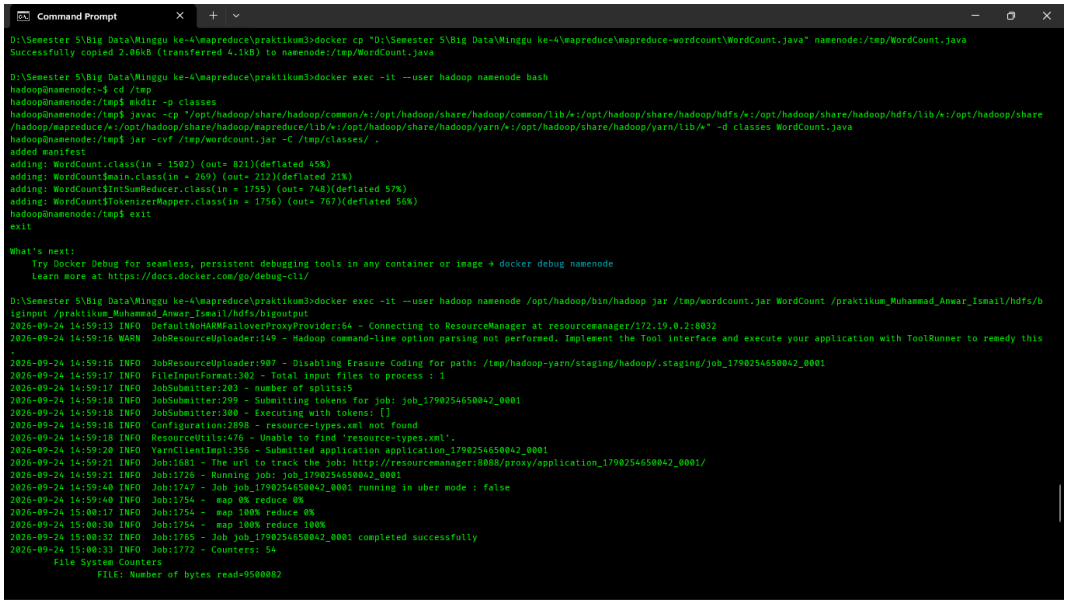


Gambar 3.1.13 - Laporan HDFS fsck untuk bigfile.txt yang memperlihatkan file terpecah menjadi 5 block secara terdistribusi pada 2 DataNode (akibat limitasi block size 1 MB)_3.png

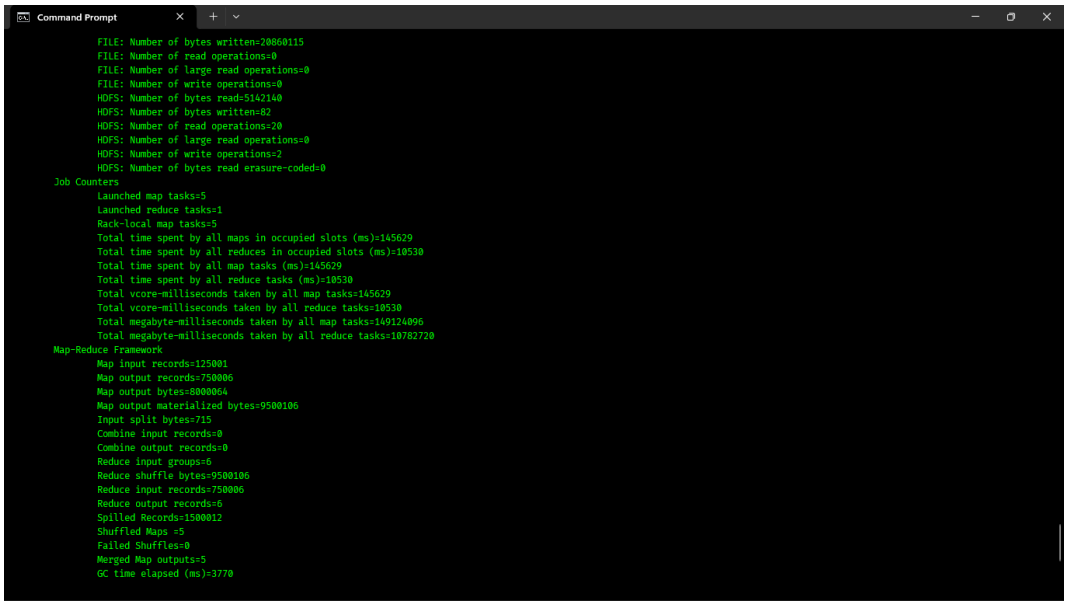
```
docker cp bigfile.txt namenode:/tmp/bigfile.txt
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput
```

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -Ddfs.blocksize=1048576 -Ddfs.replication=2 -put -f /tmp/bigfile.txt /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs fsck /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput/bigfile.txt -files -blocks -locations
```

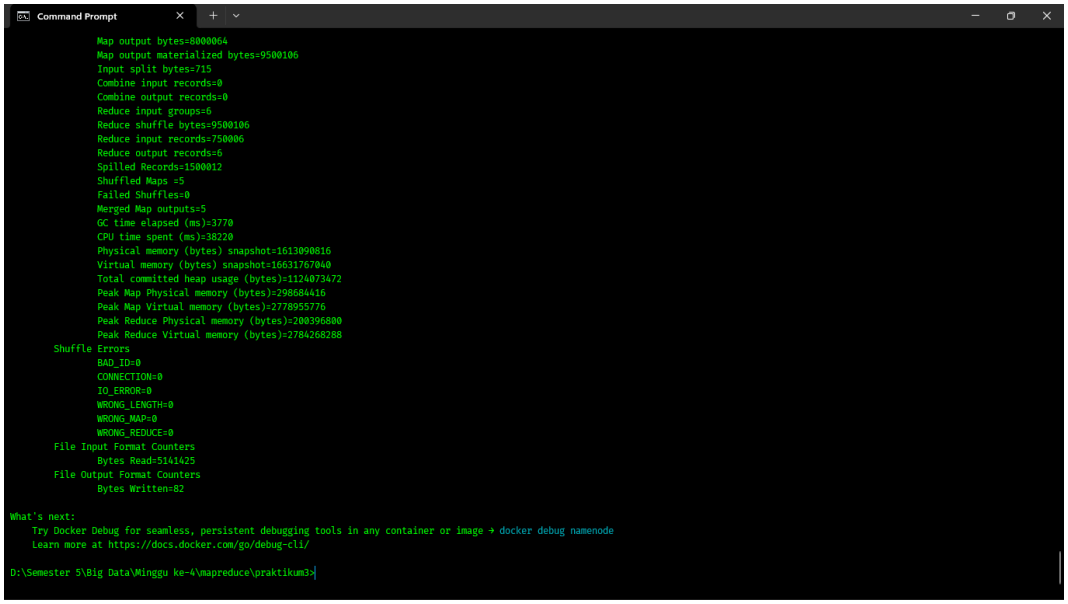
Bigfile.txt disalin ke NameNode sebesar 5,13 MB lalu diunggah dengan block size 1048576 byte dan replication factor 2, sementara kesalahan penulisan opsi `--user` pada percobaan pertama langsung diperbaiki pada perintah berikutnya. Hasil `hdfs dfs -ls` memperlihatkan replication 2 dengan ukuran 5125041 byte, dan awal laporan fsck sudah menampilkan 5 block. Rincian fsck menampilkan lima block dengan ID blk_1073741857 sampai blk_1073741861. Empat block pertama berukuran 1048576 byte, block kelima berukuran 930737 byte sebagai sisa file, dan setiap block memiliki dua replica pada 172.19.0.5 serta 172.19.0.4. Ringkasan fsck mencatat Total blocks sebanyak 5 dengan rata-rata ukuran block 1025008 B dan Average block replication 2.0. Seluruh block tercatat 100% minimally replicated tanpa missing, corrupt, maupun under-replicated blocks, dan statusnya HEALTHY.



Gambar 3.1.14 - Mendokumentasikan Application ID dan status penyelesaian (SUCCEEDED) dari job MapReduce WordCount_1.png



Gambar 3.1.15 - Menampilkan job counters dan status penyelesaian (SUCCEEDED) dari job MapReduce WordCount_2.png

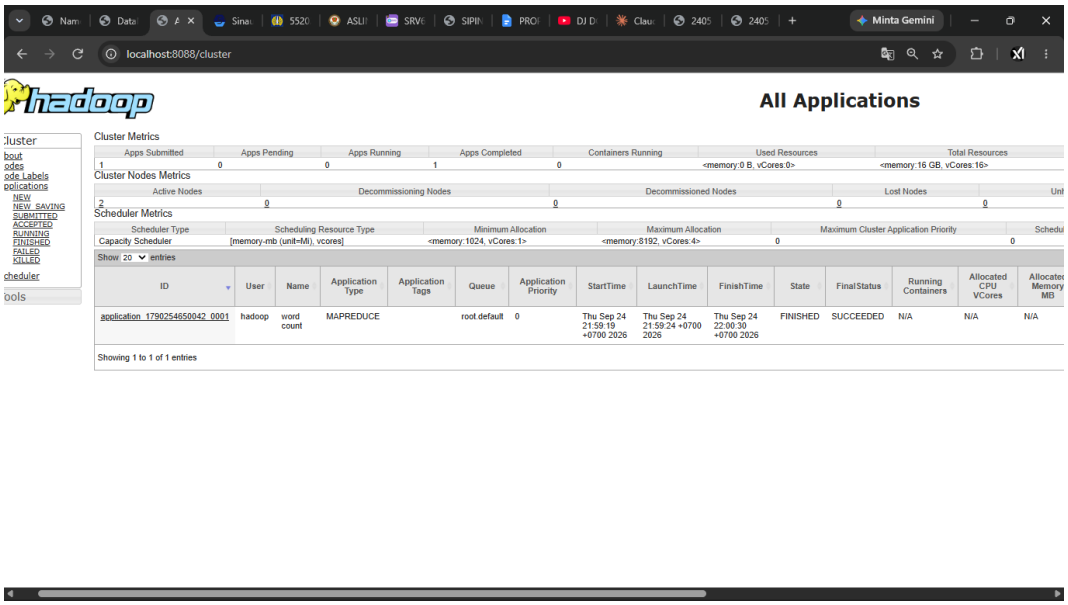


Gambar 3.1.16 - Menampilkan job counters dan status penyelesaian (SUCCEEDED) dari job MapReduce WordCount_3.png

```
docker cp "D:\Semester 5\Big Data\Minggu ke-4\mapreduce\mapreduce-wordcount\WordCount.java" namenode:/tmp/WordCount.java
docker exec -it --user hadoop namenode bash
cd /tmp
mkdir -p classes
javac -cp "/opt/hadoop/share/hadoop/common/*:/opt/hadoop/share/hadoop/common/lib/*:/opt/hadoop/share/hadoop/hdfs/*:/opt/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/mapreduce/*:/opt/hadoop/share/hadoop/mapreduce/lib/*:/opt/hadoop/share/hadoop/yarn/*:/opt/hadoop/share/hadoop/yarn/lib/*" -d classes WordCount.java
jar -cvf /tmp/wordcount.jar -C /tmp/classes/ .
exit
docker exec -it --user hadoop namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput /praktikum_Muhammad_Anwar_Ismail/hdfs/bigoutput
```

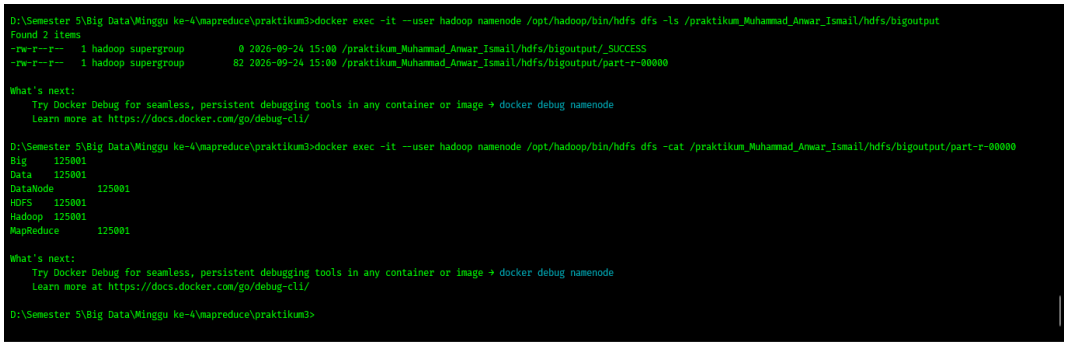
File WordCount.java dikompilasi di dalam container menjadi wordcount.jar, lalu job dijalankan pada direktori biginput. Log menunjukkan number of splits sebanyak 5, Application ID application_1790254650042_0001, dan job selesai

pada map 100% reduce 100%. Job counters mencatat 5 map task dan 1 reduce task yang dijalankan, sesuai dengan lima block pada file input. Nilai Rack-local map tasks sebesar 5 menunjukkan bahwa seluruh Mapper mendapat data dari node yang berada dalam rak yang sama. Counters lanjutan mencatat Reduce input groups sebanyak 6, Reduce output records sebanyak 6, dan Shuffled Maps sebanyak 5. Seluruh nilai Shuffle Errors bernilai 0, dan File Output Format Counters mencatat 82 bytes yang ditulis ke HDFS.



Gambar 3.1.17 - Antarmuka YARN UI.png

Antarmuka YARN pada localhost:8088 menampilkan satu aplikasi bernama word count dengan tipe MAPREDUCE, state FINISHED, dan FinalStatus SUCCEEDED. Panel Cluster Metrics menunjukkan 2 Active Nodes dengan total resource 16 GB memori dan 16 vCores.



Gambar 3.1.18 - Output akhir dari job MapReduce WordCount di HDFS menggunakan perintah hdfs dfs -cat.png

```
docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /praktikum_Muhammad_Anwar_Ismail/hdfs/bigoutput/part-r-00000
```

Direktori bigoutput berisi file _SUCCESS dan part-r-00000 berukuran 82 byte. Isi output memuat enam kata unik, yaitu Big, Data, DataNode, HDFS, Hadoop, dan MapReduce, masing-masing dengan hitungan 125001.

3.2 Rekapitulasi Block bigfile.txt

Berkas bigfile.txt berukuran 5.125.041 bytes dan tersimpan dengan block size 1 MB, sehingga terbentuk 5 block. Setiap block memiliki dua replica yang berada pada datanode1 (172.19.0.5) dan datanode2 (172.19.0.4).

Block	ID Block	Ukuran (byte)	Jumlah Replica	DataNode
0	blk_1073741857_1033	1.048.576	2	172.19.0.5 dan 172.19.0.4
1	blk_1073741858_1034	1.048.576	2	172.19.0.4 dan 172.19.0.5
2	blk_1073741859_1035	1.048.576	2	172.19.0.5 dan 172.19.0.4
3	blk_1073741860_1036	1.048.576	2	172.19.0.5 dan 172.19.0.4

4	blk_1073741861_1037	930.737	2	172.19.0.4 dan 172.19.0.5

3.3 Catatan Ukuran bigfile.txt

Ukuran bigfile.txt tercatat 5.125.041 bytes, lebih besar 41 byte dari perhitungan 125.000 baris dengan panjang 41 byte per baris. Selisih tersebut setara dengan satu baris tambahan, yang kemungkinan berasal dari eksekusi perintah >> pada file yang sudah terisi satu baris sebelumnya. Kondisi ini konsisten dengan hasil WordCount, karena setiap kata tercatat sebanyak 125.001 kali.

3.4 Hubungan DataNode, NodeManager, Mapper, Reducer, dan ResourceManager

Pada percobaan ini, DataNode menyimpan block bigfile.txt pada dua node, yaitu datanode1 dan datanode2. ResourceManager menerima job WordCount dengan Application ID application_1790254650042_0001 dan mengatur pembagian sumber daya ke seluruh klaster. NodeManager pada nodemanager1 dan nodemanager2 kemudian menyediakan container tempat task berjalan.

Sebanyak 5 Mapper dijalankan di dalam container tersebut, satu Mapper untuk setiap block, dan setiap Mapper membaca salah satu replica dari DataNode. Hasil kerja Mapper dikirim melalui tahap shuffle ke 1 Reducer, yang menjumlahkan kemunculan setiap kata lalu menuliskan hasilnya ke HDFS pada file part-r-00000. Counter Rack-local map tasks bernilai 5 menunjukkan bahwa seluruh Mapper membaca data dari node dalam rak yang sama. Container NodeManager dan DataNode pada klaster ini berjalan terpisah, sehingga kondisi rack-local menjadi hasil yang wajar.

3.5 Alasan Replica Tidak Berarti Data Diproses Dua Kali

Replica adalah salinan block yang disimpan untuk menjaga ketersediaan data. Hadoop membuat satu Mapper untuk setiap input split, sehingga satu block yang memiliki dua replica tetap diproses oleh satu Mapper. Hal ini terbukti dari job WordCount yang meluncurkan 5 map task untuk 5 block, meskipun jumlah replica seluruhnya 10.

Mapper cukup membaca satu replica yang tersedia. Replica kedua baru dipakai ketika replica pertama tidak dapat diakses, misalnya karena DataNode mati. Hasil WordCount yang menunjukkan setiap kata muncul 125.001 kali, sesuai isi file, menjadi bukti bahwa tidak ada data yang terhitung ganda.

BAB IV
KESIMPULAN

4.1 Kesimpulan

Praktikum ini berhasil menunjukkan cara HDFS menyimpan berkas sebagai block dan replica pada klaster Hadoop 3.4.3 berbasis Docker dengan dua DataNode aktif. File input.txt berukuran 132 bytes tersimpan sebagai 1 block dengan 2 replica pada datanode1 dan datanode2, sedangkan bigfile.txt berukuran 5.125.041 bytes terpecah menjadi 5 block setelah block size diatur menjadi 1 MB, dengan setiap block memiliki 2 replica. Nilai default dfs.replication sebesar 1 dan dfs.blocksize sebesar 128 MB menjelaskan alasan pengaturan manual diperlukan agar kedua konsep tersebut dapat diamati. Job WordCount pada bigfile.txt menjalankan 5 map task dan 1 reduce task, selesai dengan status SUCCEEDED pada Application ID application_1790254650042_0001, dan menghasilkan enam kata unik dengan hitungan masing-masing 125.001. Nilai Rack-local map tasks sebesar 5 memperlihatkan penerapan data locality pada tingkat rak, dan Jumlah map task yang sama dengan jumlah block, yaitu 5, memperlihatkan bahwa setiap block diproses oleh satu Mapper saja, sedangkan replica kedua tersimpan sebagai cadangan untuk fault tolerance. Kendala kecil berupa kesalahan urutan opsi --user pada saat membuat direktori biginput menjadi pelajaran bahwa ketelitian penulisan perintah sangat memengaruhi kelancaran praktikum.

LAMPIRAN A
COMMAND UTAMA

No	Command	Fungsi
1	docker compose up -d	Menjalankan seluruh container Hadoop di latar belakang
2	docker ps	Memeriksa container yang sedang berjalan
3	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfsadmin -report	Menampilkan laporan DataNode aktif dan kapasitas HDFS
4	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs getconf -confKey dfs.replication	Memeriksa nilai replication factor default
5	mkdir praktikum3	Membuat folder kerja di Windows
6	echo Hadoop HDFS DataNode MapReduce Big Data>input.txt echo Hadoop stores data as blocks and replicas>>input.txt echo Hadoop MapReduce processes data stored in HDFS>>input.txt	Membuat file input.txt berisi tiga baris teks
7	dir input.txt	Menampilkan ukuran file input.txt di Windows

8	docker cp input.txt namenode:/tmp/input.txt	Menyalin input.txt ke container NameNode
9	docker exec -it --user hadoop namenode ls -lh /tmp/input.txt	Memverifikasi file di dalam container
10	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -mkdir -p /praktikum_Muhammad_Anwar_Ismail/hdfs/input	Membuat direktori input di HDFS
11	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -Ddfs.replication=2 -put -f /tmp/input.txt /praktikum_Muhammad_Anwar_Ismail/hdfs/input	Mengunggah input.txt ke HDFS dengan replication factor 2
12	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum_Muhammad_Anwar_Ismail/hdfs/input	Menampilkan isi direktori HDFS beserta replication
13	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs fsck /praktikum_Muhammad_Anwa	Menampilkan block dan lokasi replica input.txt

	r_Ismail/hdfs/input/input.txt -files -blocks -locations	
14	for /L %i in (1,1,125000) do @echo Hadoop HDFS DataNode MapReduce Big Data>>bigfile.txt	Membuat bigfile.txt sebanyak 125.000 baris
15	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs getconf -confKey dfs.blocksize	Memeriksa ukuran block default
16	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -Ddfs.blocksize=1048576 -Ddfs.replication=2 -put -f /tmp/bigfile.txt /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput	Mengunggah bigfile.txt dengan block size 1 MB dan replication 2
17	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs fsck /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput/bigfile.txt -files -blocks -locations	Menampilkan 5 block dan lokasi replica bigfile.txt
18	javac -cp "/opt/hadoop/share/hadoop/c	Mengompilasi WordCount.java dan membuat file JAR

	ommon/*:/opt/hadoop/share/hadoop/common/lib/*:/opt/hadoop/share/hadoop/hdfs/*:/opt/hadoop/share/hadoop/hdfs/lib/*:/opt/hadoop/share/hadoop/mapreduce/*:/opt/hadoop/share/hadoop/mapreduce/lib/*:/opt/hadoop/share/hadoop/yarn/*:/opt/hadoop/share/hadoop/yarn/lib/*" -d classes WordCount.java jar -cvf /tmp/wordcount.jar -C /tmp/classes/ .	
19	docker exec -it --user hadoop namenode /opt/hadoop/bin/hadoop jar /tmp/wordcount.jar WordCount /praktikum_Muhammad_Anwar_Ismail/hdfs/biginput /praktikum_Muhammad_Anwar_Ismail/hdfs/bigoutput	Menjalankan job WordCount pada bigfile.txt
20	docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -cat /praktikum_Muhammad_Anwa	Melihat isi hasil WordCount

	r_lsmail/hdfs/bigoutput/part-r-00000	

LAMPIRAN B
PORT YANG DIGUNAKAN

Layanan	Port Host dan Container	Fungsi
NameNode (HDFS Web UI)	9870	Mengakses HDFS Web UI lewat browser
NameNode (RPC)	8020	Komunikasi internal client dengan HDFS
ResourceManager (YARN Web UI)	8088	Mengakses YARN Web UI dan memantau job MapReduce
NodeManager 1	8042	Web UI NodeManager 1
NodeManager 2	8043	Web UI NodeManager 2 (port host digeser ke 8043 supaya tidak bentrok dengan NodeManager 1)